

TILLÄMPAD ARTIFICIELL INTELLIGENS

F5 - NAVIGATION

FEEDBACK PÅ KOD

- Casts

- Vad gör reinterpret cast?
- Vad gör dynamic cast?
- Vad gör static cast?

- Vanligt misstag

- Att alltid använda dynamic cast för pointers
- Om du **vet** att en down cast är safe, använd static cast.
- Dynamic cast behövs bara om du behöver checka resultatet av casten.

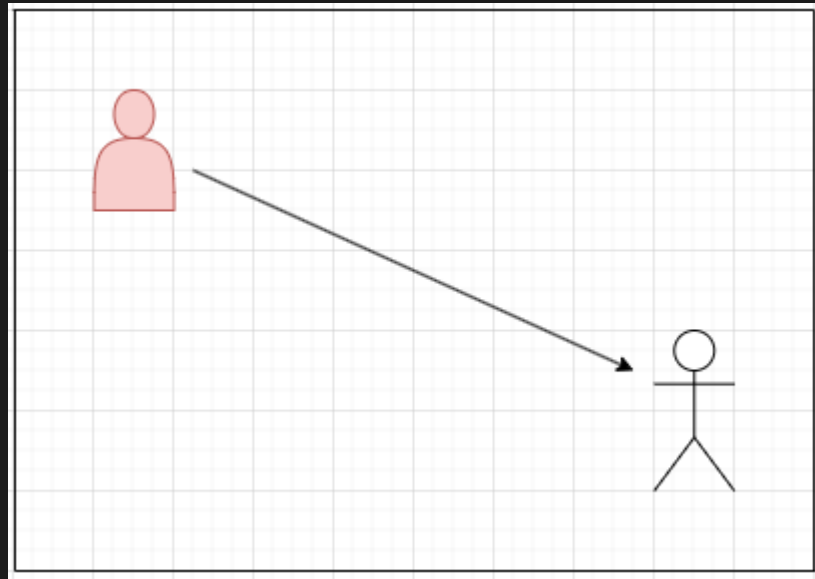
INTRO

- I början av kursen snackade vi om movement
 - Steering behaviours, etc...
- Och det var ju najs
 - ... men vi har än så länge bara rört oss i en void.
 - Från spelprojekten i ettan kanske ni mest har använt grid-baserad movement.
 - En tile har mylsWalkable, eller nåt liknande.
 - Men hur representerar vi en mer komplex spel-värld på ett sätt som AI smidigt kan röra sig på?
 - Det ska vi snacka om idag!



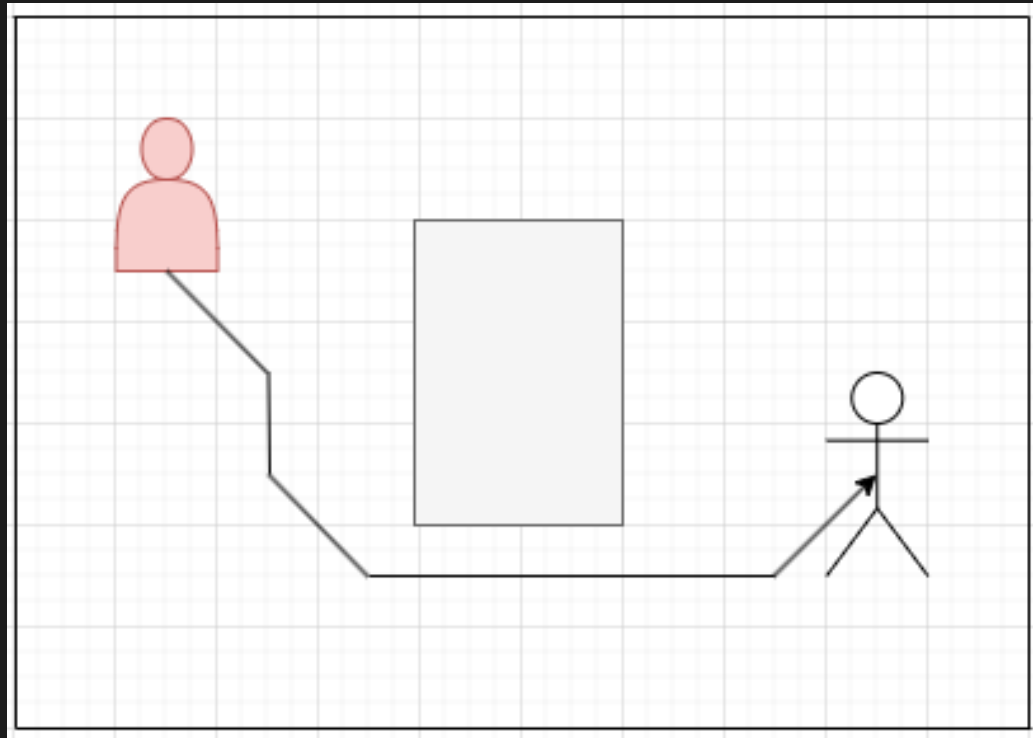
INTRO

- Ett av de vanligaste fallen:
 - Vi vill att en fiende ska ta sig till spelaren
- Vi kan säga åt fienden att gå rakt mot spelaren
 - Funkar om spelplanen är tom



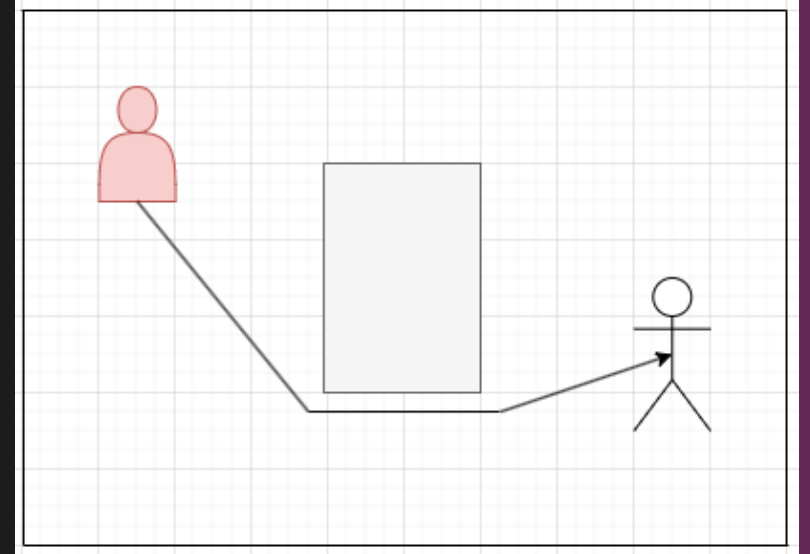
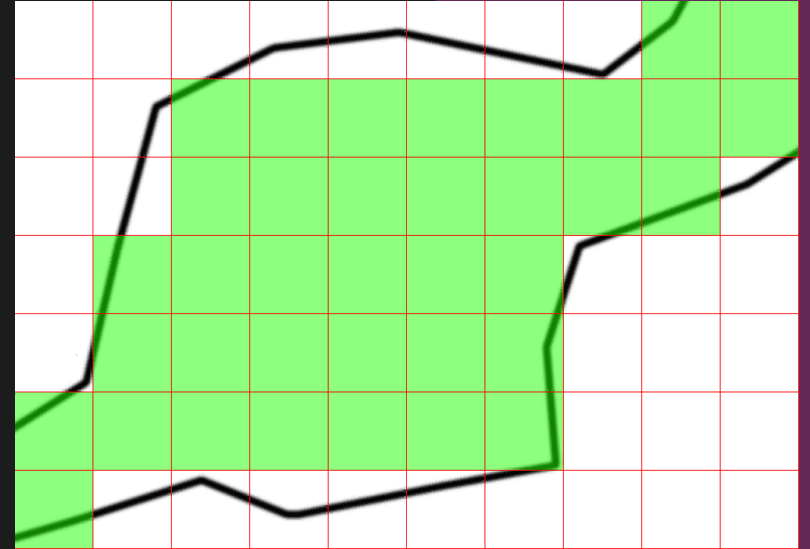
INTRO

- Om vi har ett hinder?
 - No problemo, vi använder A^*
 - ... but it was all problemo



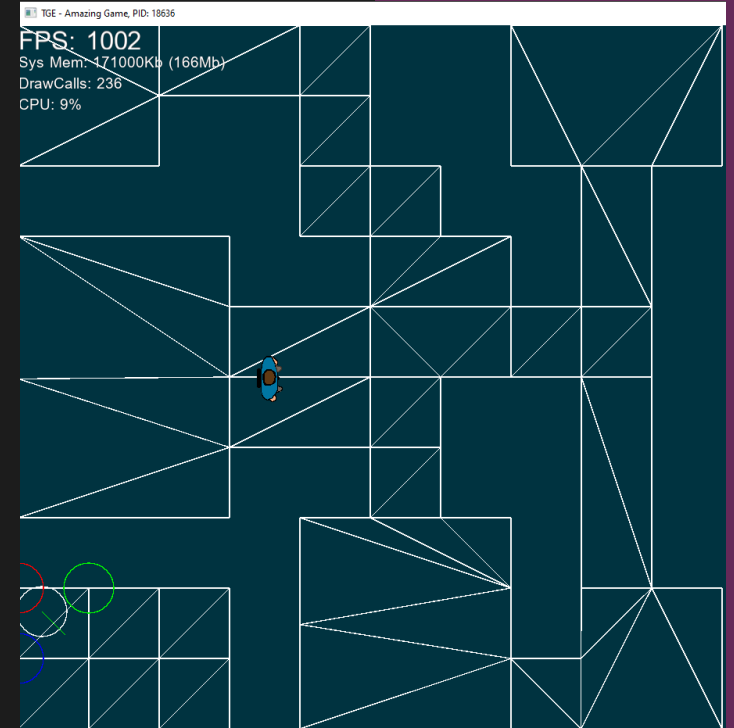
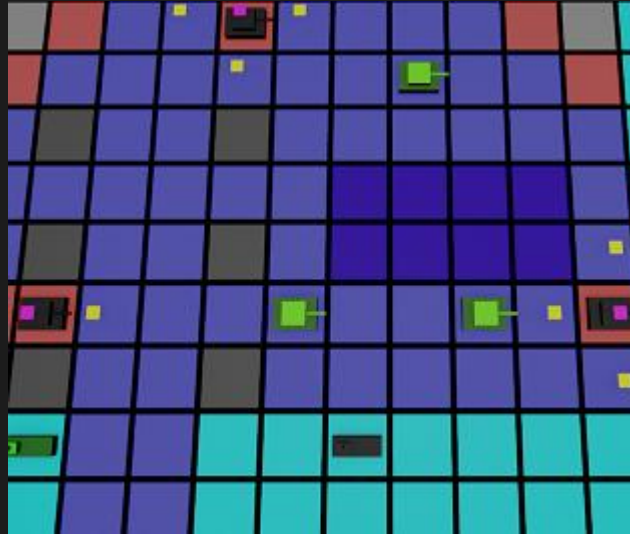
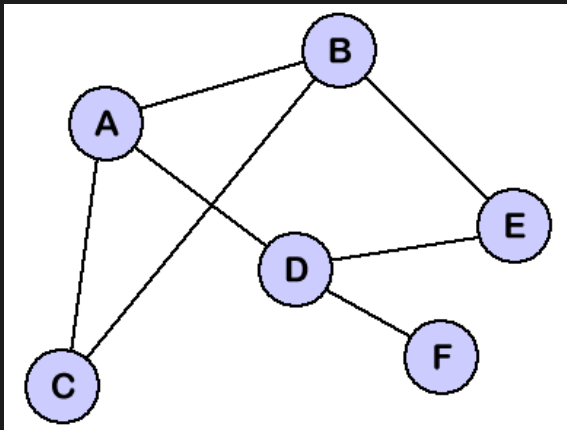
INTRO

- Vad gör vi om världen inte kan representeras med en grid?
- Vad gör vi för att få en " snyggare " path till spelaren?
- *Vårt mål är att röra AI:n kortaste vägen från A till B i en spelvärld med komplex geometri och med en path som inte ser fejk ut.*
 - What to do?



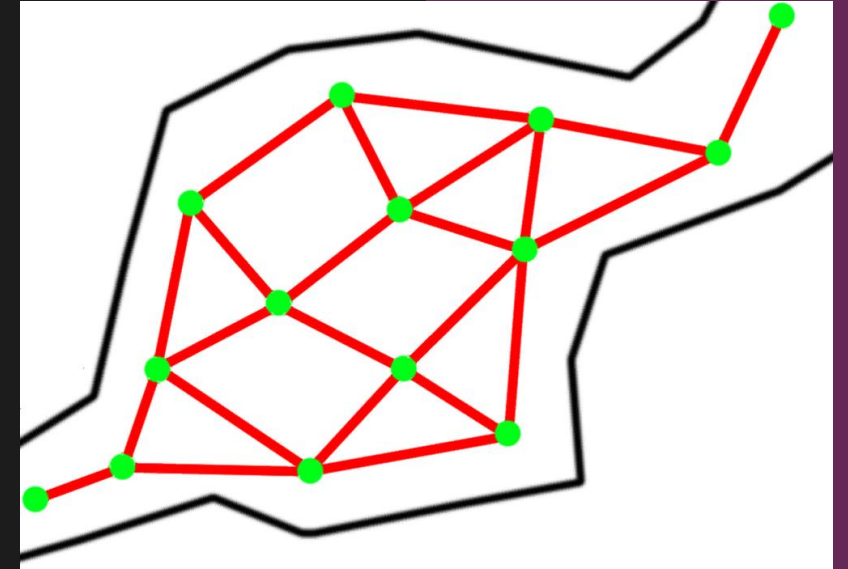
NAVIGATION GRAPHS

- Vi vill använda pathfinding
 - Men pathfinding behöver nån typ av graf att arbeta på
 - För de flesta spel måste vi generera vår egna navigation graph.
- Finns lite olika typer vi ska kolla på nu:
 - Nodgraf
 - Grid
 - Navmesh



NODGRAF

- Detta är den simplaste navigationsmetoden, och den som är mest idiotsäker.
 - Ganska enkel att göra
 - Vi skapar ett gäng noder
 - Kopplar ihop dem med edges
- Vi ger ett verktyg till våra LDs där de kan lägga ut noder och koppla ihop dem.
 - LD kan då kontrollera vilka delar av världen de vill att vår AI ska kunna gå till.
- Alltid bra att fundera på om vi kan komma undan med en simpel nodgraf eller inte.
 - Billigare pathfinding och mindre jobb.
 - (för oss kodare iaf)



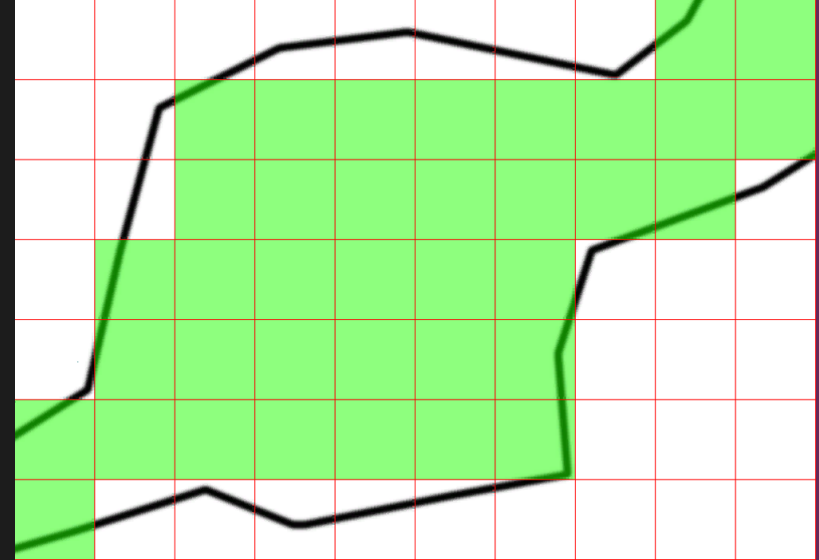
(sidenode: edge, connection, portal, arc, link, line... same shit)

NODGRAF

- Måste skapas av en designer
 - Väldigt svårt att göra ett verktyg som automatiskt genererar noder och edges.
 - Dock är grafen helt buggfri om designern skapar den "rätt".
 - Vi vet ju att allt blir perfekt eftersom en designer har sagt att det är ok för AI:n att gå till noderna. Designern "hårdkodar" AI:ns movement.
- AI:n måste byggas runt noderna
 - Noder kan ha olika flags/properties
 - Cover, HealStation, etc...
 - Designers kan sätta dessa flaggor och data på noderna som i sin tur påverkar vår pathfinding.

GRID

- Man kan säga att en grid är ett subset av navmesh.
 - Med andra ord, en grid är en slags navmesh.
 - Alla ytor i denna mesh ser likadana ut.
 - Väldigt enkelt att optimera.
- Kan ha olika former
 - Vi tänker oss oftast kvadrater
 - Kan vara hexagoner, trianglar, etc...
 - Vilken typ av geometrisk form som helst så länge det går att göra ett upprepande mönster av den.

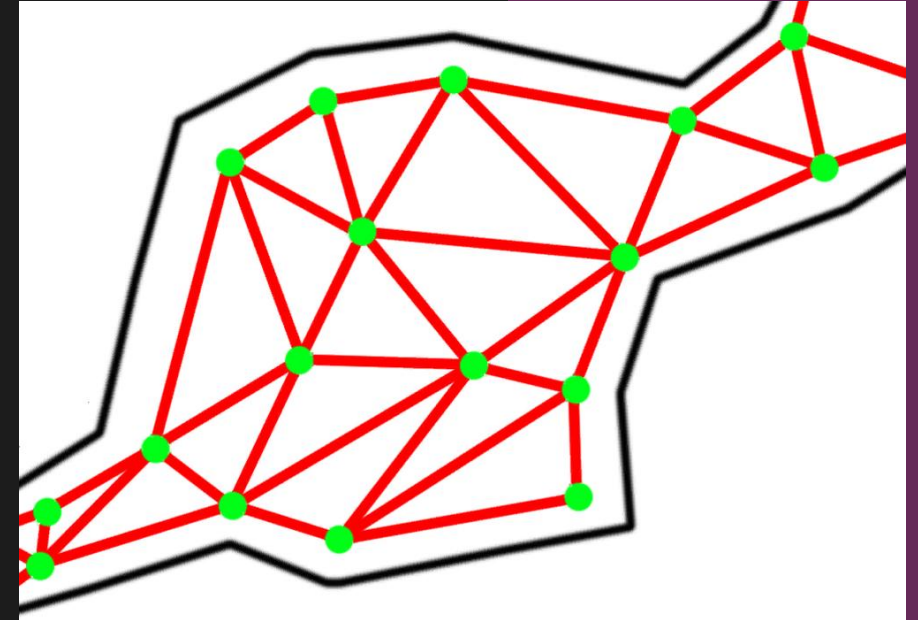


GRID

- Oftast använt när spelet är tilebaserat.
 - Då får vi ju vår navigation "gratis" från spelvärlden.
- Går att använda grid även i andra spel, men kan bli clunky.
 - Det kan uppkomma småfel.
 - T.ex. att grid:en missar mindre detaljer i geometrin.
- Kan vi inte bara ha fler och mindre tiles?
 - Jo, men det gör att vår pathfinding blir långsammare!
 - Vi kan också göra någon slags post process på grid:en.
 - Typ flytta mittpunkten på de noderna som är nära/förbi kanten av spelvärlden.
 - Men herregud vilket pill. Man blir ju mörkrädd. Vill knappt tänka på det.

NAVMESH

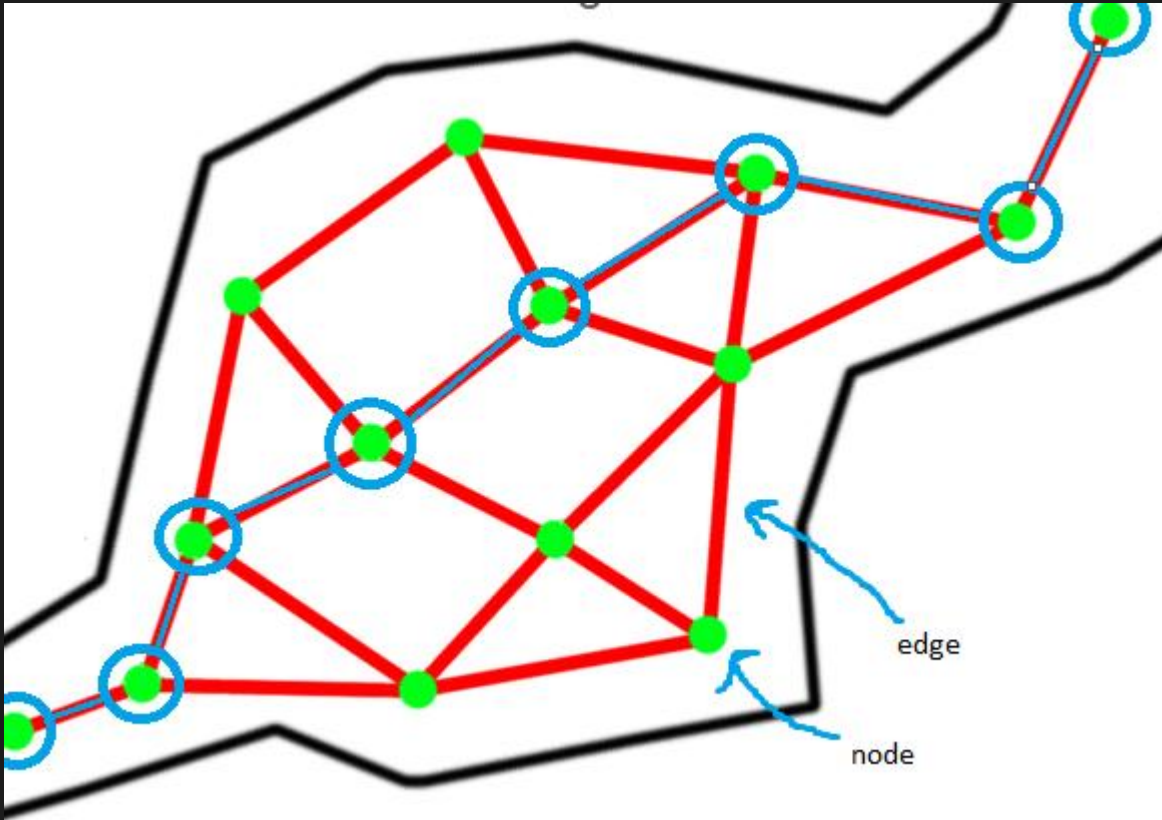
- Navmesh är the shit
- En navmesh är en graf uppbyggd av polygoner.
 - Är väldigt ofta n-gons, alltså polygoner med mer än tre vertices.
 - Det enklaste är dock att bara jobba med trianglar.
- Men hold the phone...
 - Det där ser ju typ ut som en nodgraf...?



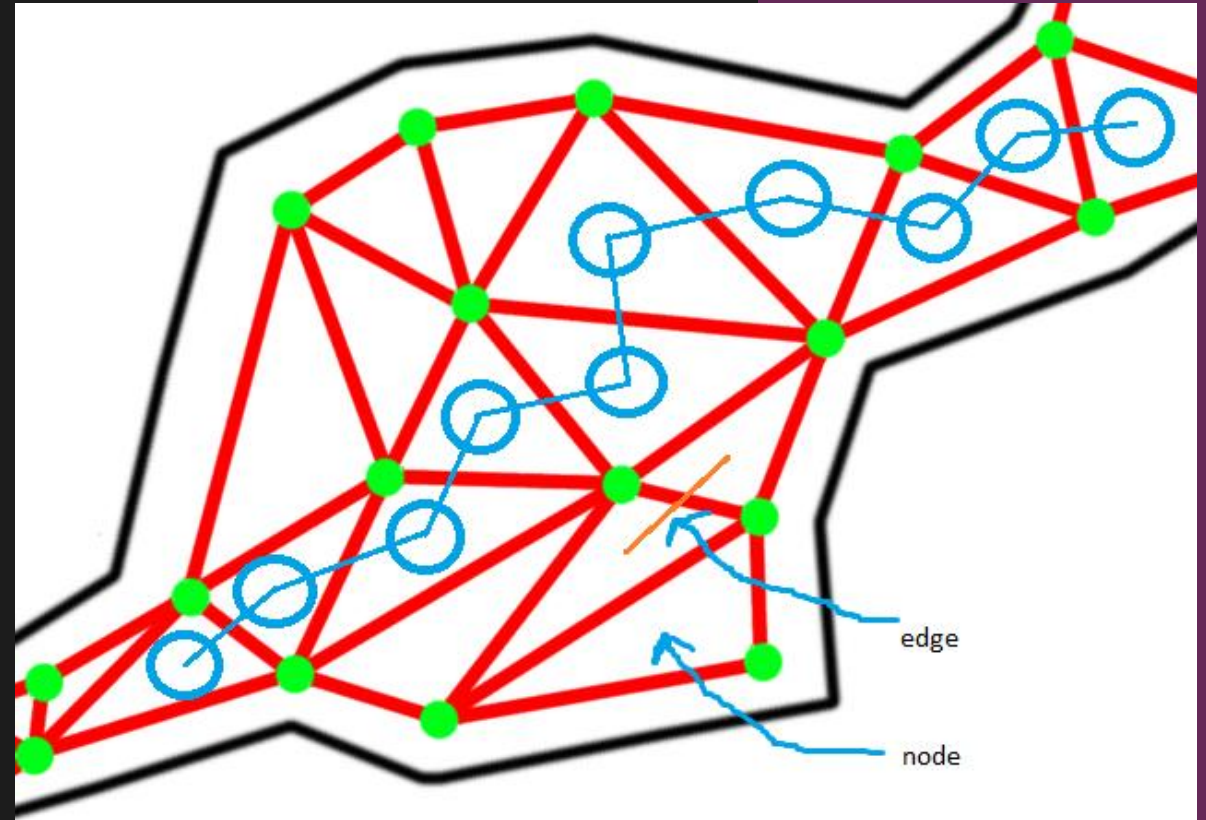
NAVMESH

- Skillnaden på nodgraf och navmesh:

Nodgraf



Navmesh

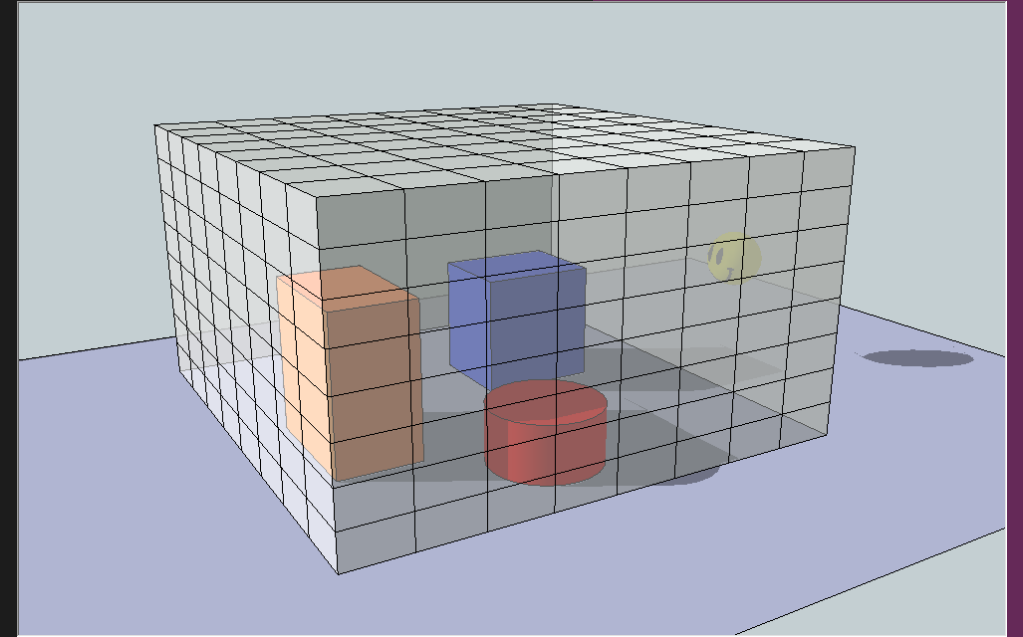


NAVMESH

- Ofta sedd som en "catch all" för spelutveckling.
 - Används i väldigt många olika typer av spel.
- Alla stora motorer har inbyggt stöd för att generera navmeshes.
- När ni bygger levlar i unreal/unity så kan de outputta en navmesh till er.
 - Det är möjligt att skapa en navmesh för hand.
 - Det är ju en "vanlig" mesh. Men vem har lust att edita den varje gång leveln ändras lite?

NAVMESH

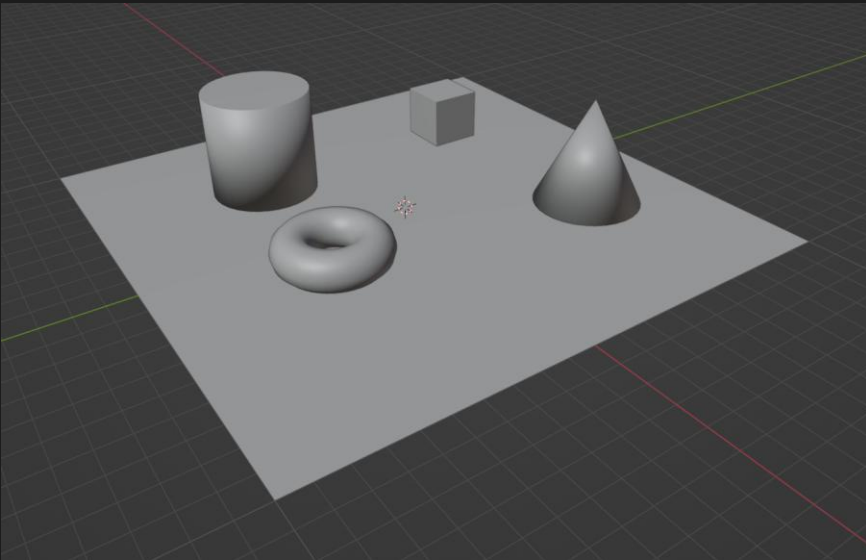
- Så hur genereras en navmesh?
- Inte helt straight forward
- Brukar ske med hjälp av något som heter voxelisering.
- Vi skapar en volym som innehåller level-geometri.
 - Ett rätblock i 3D som överlappar en del av vår värld
- Denna volymen voxeliseras, dvs. delas in i voxlar
 - Voxel = volympixel, eller 3D-pixel
 - En 3-dimensionell grid
- För varje voxel undersöker vi vad som finns i världen.



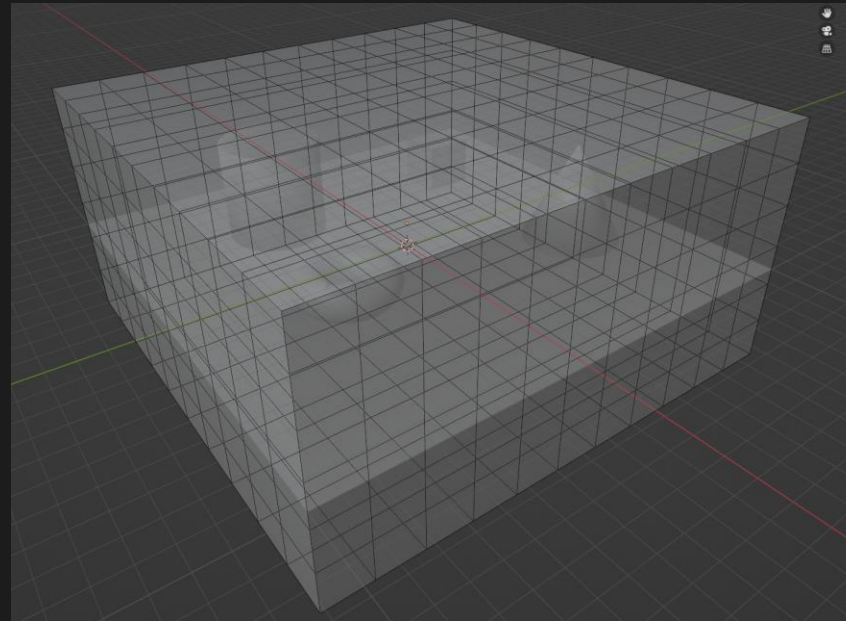
NAVMESH

- Med hjälp av våra voxlar kan vi analysera geometrin i varje voxel och i slutändan generera ett plan av trianglar.

1. Vår level

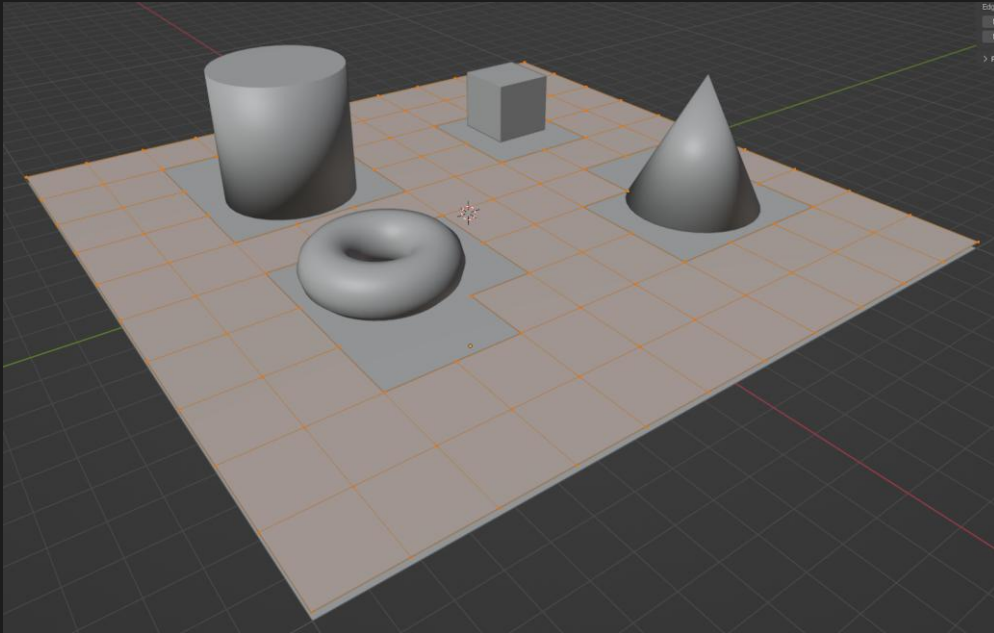


2. Voxelisering

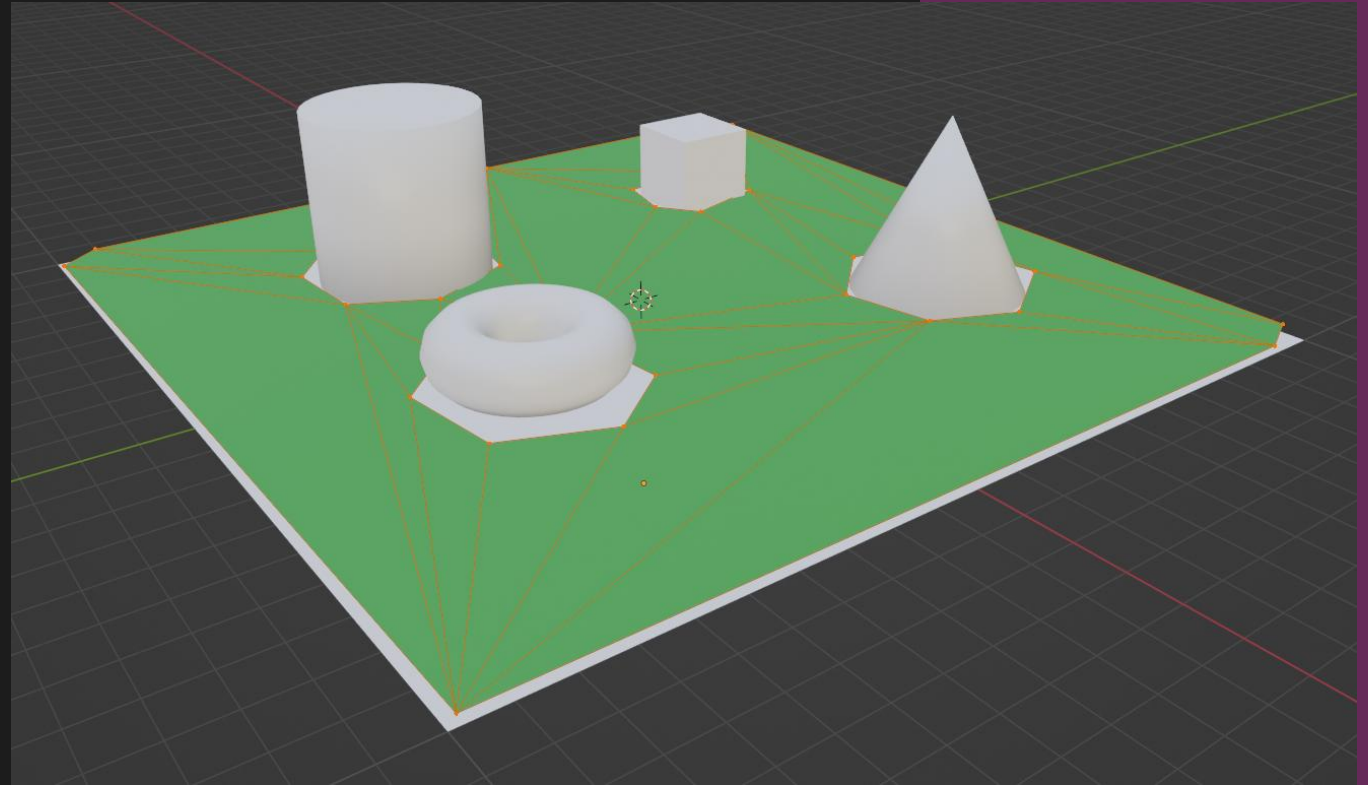


NAVMESH

3. En grid som saknar vertiser där det finns level-geometri

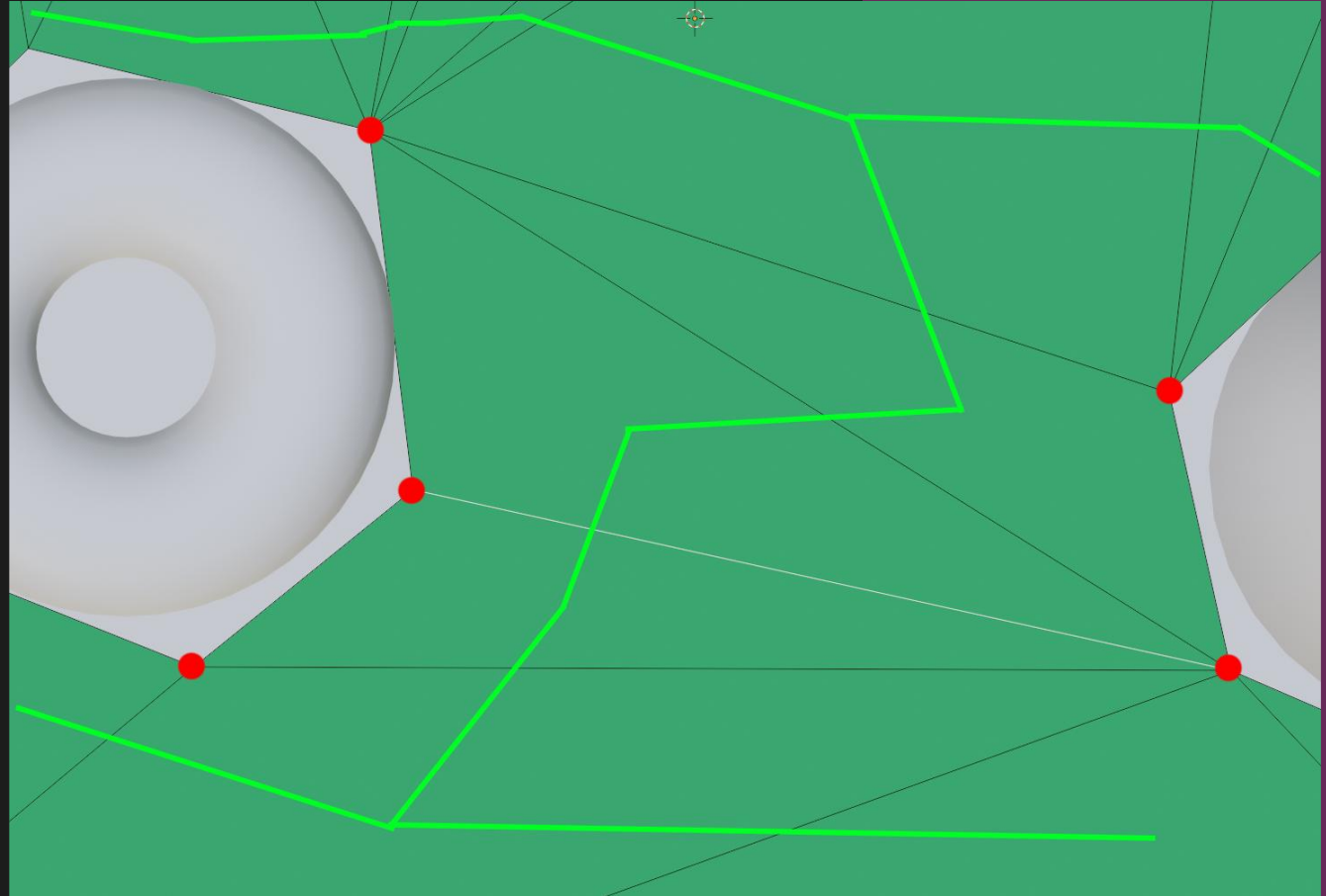


4. "Förfining" av vår grid



NAVMESH

- Recap:
- Vårt plan som vi får resulterar i en graf.
 - Varje polygon är en nod.
 - Varje kant på polygonen är en edge.
- Om två polygoner delar en kant så är de ihopkopplade.



NAVMESH

- Vi vill nog inte ha hela vår värld i en och samma voxel volume.
 - Då får navmesh-genereringen en massa tomma ytor att gå igenom.
- Istället placerar vi ut flera volymer som täcker ytan på vår värld som vi vill kunna navigera på.
- Resulterar i flera navmeshes
 - Alltså kan vi behöva koppla ihop olika navmeshes.
 - Navmeshes kan bli noder i en graf som beskriver hur navmeshes är connectade. Navmesh-ception.
 - När vi navigerar på en navmesh så tar vi också in i beräkningarna noder från connectade navmeshes.
 - Bara en idé, kanske inte behöver vara så invecklat.

NAVMESH

- Unity / Unreal / Recast gör navmesh generation åt er.
- Navmeshen som skapas är en fbx, en mesh.
 - Har vertiser, indexes, osv
 - Som vi är vana vid från "vanliga" meshes.

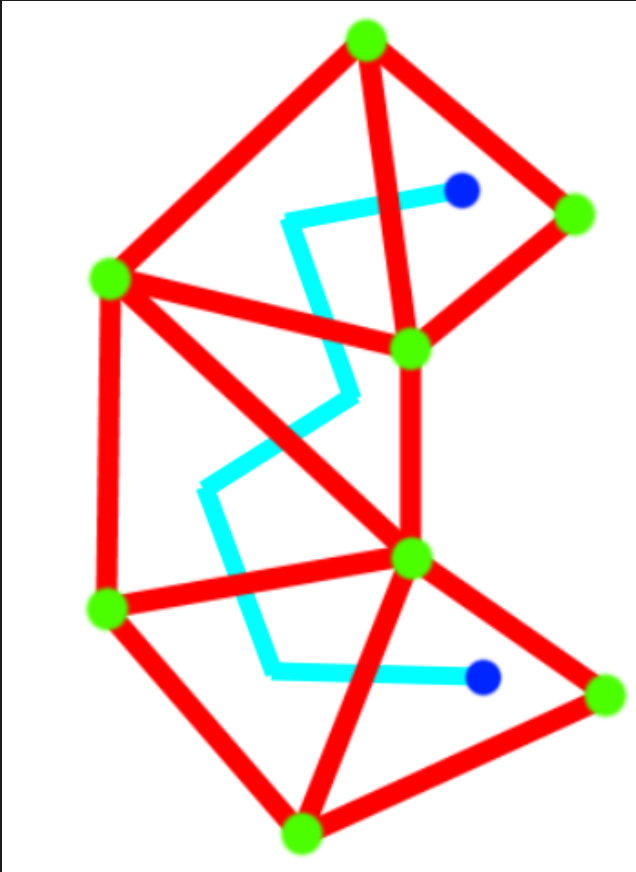
PAUS



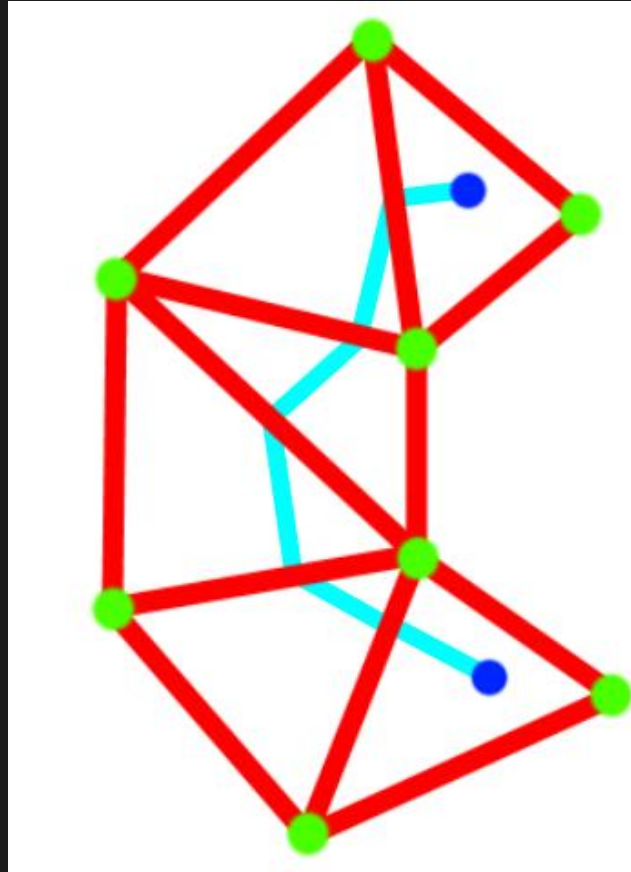
PATHFINDING

- Hur navigerar vi sen på detta då?

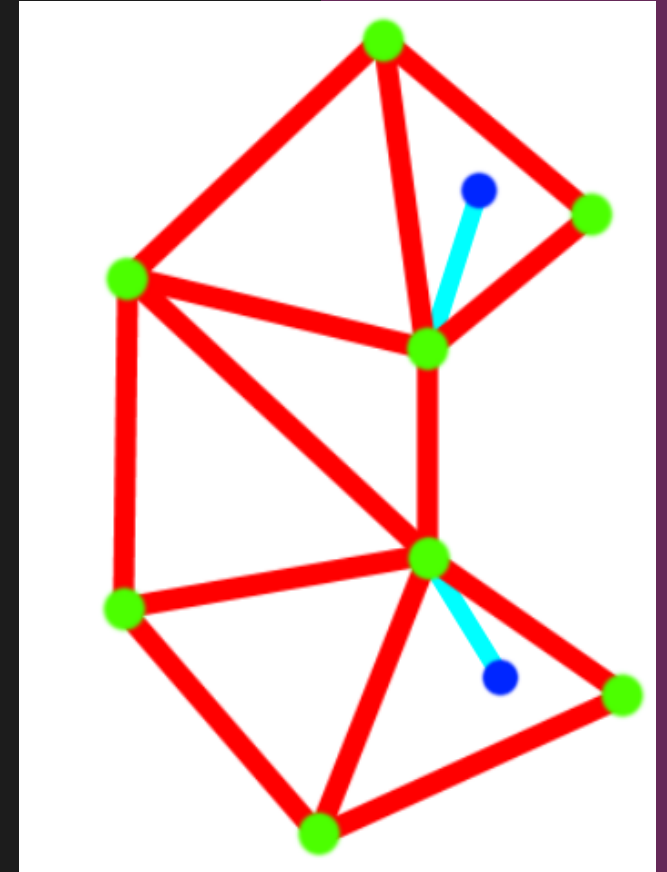
Navigering mellan
noders mittpunkter



Navigering mellan
closest point on line



Navigering mellan
vertex positions



PATHFINDING

- När vi pathfind:ar i en navmesh är det vanligt att vi använder avståndet mellan noderna som kostnad.
 - Dvs. som g-värde i vår A^* .
- Förutom det så pathfind:ar vi med vår A^* som vanligt.
 - Finns ju egentligen ingenting med A^* som hindrar oss från att använda den på något annat än en grid.

NAVMESH CAVEATS

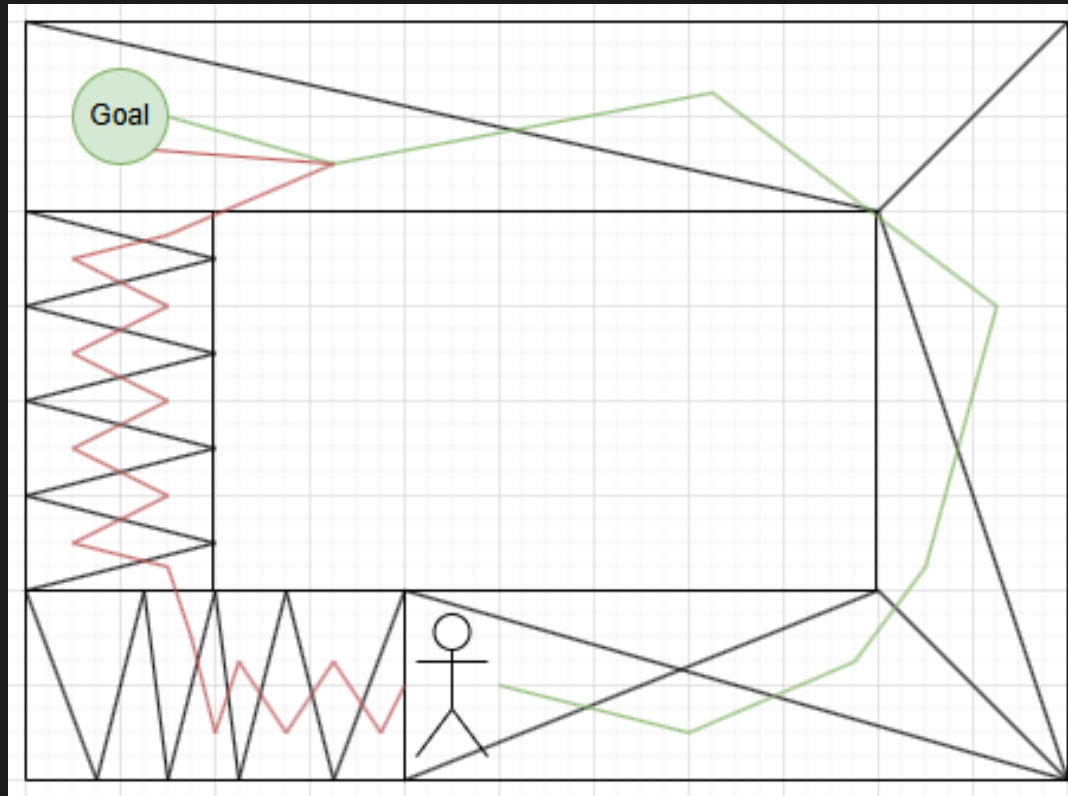
- Det finns en del drawbacks med navmeshes.
 - T.ex. är de svåra att modifiera i runtime.
- För att ändra på dem måste vi klippa i polygoner och kanter och hålla på.
- Vissa av dessa problem går att jobba runt genom att istället ge noderna på navmeshen flaggor eller höga kostnader så de inte används.
 - T.ex. `myIsTraversable = false;`
 - Eller `myNavigationCost = UINT_MAX;`

NAVMESH CAVEATS

- Ett annat problem är om våra trianglar ser wonky ut.
 - T.ex. om vi får väldigt olika storlek på trianglarna.
 - Eller om vi får s.k. “slivers”, väldigt långa och tunna trianglar.
 - Eller trianglar med vinklar på över 90 grader.
- Bäst är om alla trianglar är
 - Ungefär samma storlek
 - Inte jättetunna
 - Har vinklar under 90 grader
- Som ni ser finns det massa småpill som kan vara fel med vår navmesh.
 - Därför är navmesh generation ett rätt stort ämne.
 - Och ibland kan man t.o.m., if all else fails, behöva putsa och tweeka den manuellt efter att den är genererad.
- Det går att kompensera för triangel-problemen i våra pathfinding-algoritmer, men då blir det dyrare att köra dem.
- Ofta bättre att se till att navmeshen ser bra ut.

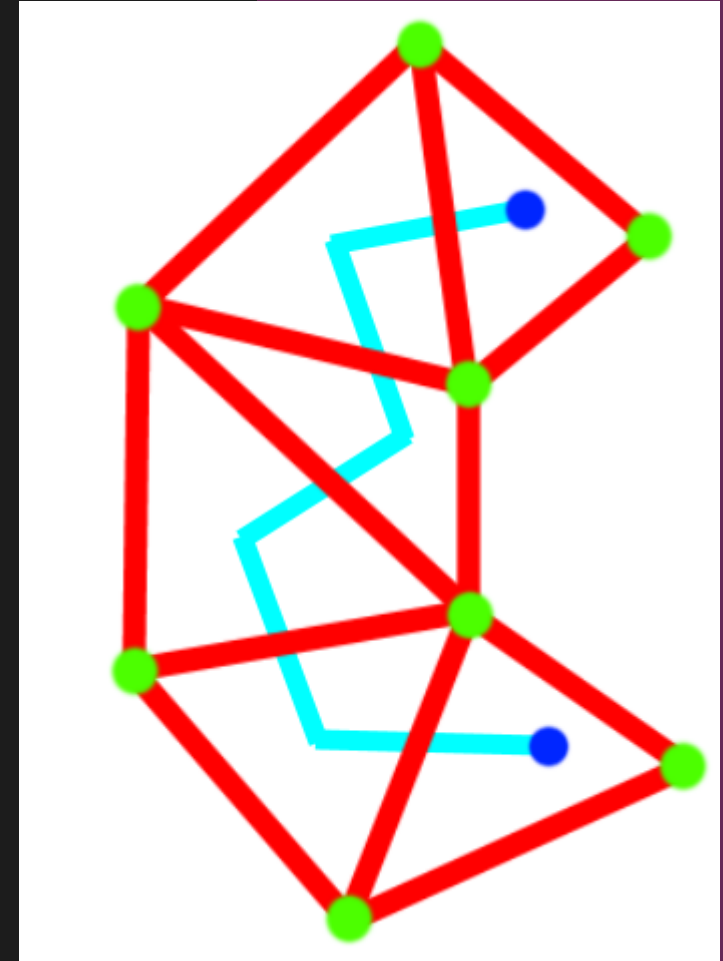
NAVMESH CAVEATS

- Exempel
 - A* tror att den kortare vägen är lägre eftersom triangelarna får det att se ut så.



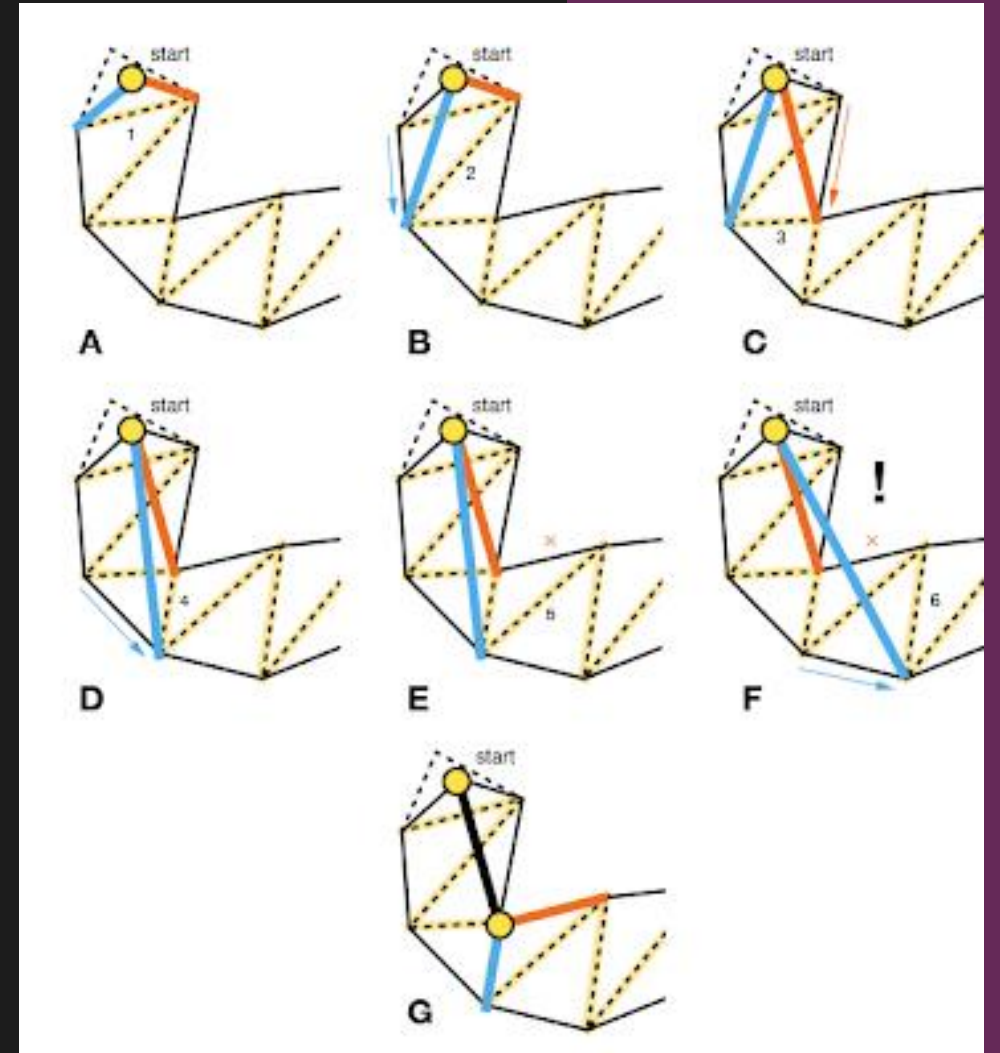
FUNNELING

- Om vi rakt av använder vägen som vår pathfinding hittar kommer vi gå från mittpunkt till mittpunkt.
 - Gör nog att en AI ser ganska puckad ut.
- För att lösa detta måste vi fixa till vår path.
 - Ta bort onödiga punkter
 - Lägga till punkter som gör att vägen ser bättre ut
 - Ofta används en s.k. "funnel" algoritm.
- Kan vara en del av pathfindingen eller som en pathfinding's "postprocess".



FUNNELING

- Vad är funneling? Och varför heter det ens funneling?
- Algoritmen för funneling ser ut något så här:
 - A: Vår startpunkt. Vi drar en vektor till högra vertisen (blå) av kommande portal och en till den vänstra (orange).
 - B: Om vi flyttar vänster/höger till nästa portal, är vi en "tratt" och är "tratten" mindre än den förra? Ja.
 - C: Flytta andra checken också till nästa portal, är vi en "tratt" och är "tratten" mindre än den förra? Ja.
 - D: Testa flytta checken till nästa portal, är vi en "tratt" och är "tratten" mindre än den förra? Ja.
 - E: Testa flytta checken till nästa portal, är vi en "tratt" och är "tratten" mindre än den förra? Nej!
 - Då behåller vi den gamla vertisen där vi fortfarande var en krympande "tratt".
 - F: Vi testar flytta den andra checken, är vi fortfarande en "tratt" och är "tratten" mindre än den förra?
 - Nej, vi ligger på fel sida om den andra vektorn!
 - G: Då sätter vi en ny startpunkt och fortsätter algoritmen därifrån. Denna punkt kallas vår nuvarande apex-punkt.



FUNNELING

- Detta är bara den överhängande idén med funneling.
 - Finns såååå många edge cases
 - Och inte särskilt mycket info online
 - Så därför får ni funneling av mig!
 - Mer om det när vi snackar om uppgiften
- Sidenote
 - Ibland kallas funneling också för string pull
 - Effekten av funneling är som om man har en tråd som är fast i path start och path end och så drar man i båda ändarna.

```
void NavMesh::PerformFunneling(NavMeshPath& outputPath, const H3_Vector2_F32 startPosition, const H3_Vector2_F32 endPosition, const H3_GrowingArray<const Node*>& nodes) const
{
    const Node* startNode = nodes[0];
    const Node* endNode = nodes[nodes.GetSize() - 1];

    if (startNode == endNode)
    {
        outputPath.AddPoint(startPosition);
        outputPath.AddPoint(endPosition);
        return;
    }

    const SizeType nOfNodes = nodes.GetSize();
    H3_GrowingArray<Portal*> portals;
    portals.Reserve(nOfNodes - 1);

    for (uint i = 0; i < nOfNodes - 1; i++)
    {
        const Portal portal = GetPortalBetweenNodes(nodes[i], nodes[i + 1]);
        portals.PushBack(portal);
    }

    portals.PushBack([ endPosition, endPosition ]);

    outputPath.AddPoint(startPosition);
    H3_Vector2_F32 apex = startPosition;
    uint leftIndex = 0;
    uint rightIndex = 0;

    const SizeType nOfPortals = portals.GetSize();
    for (uint i = 1; i < nOfPortals; i++)
    {
        if (portals[i].myLeft != portals[leftIndex].myLeft)
        {
            const H3_Vector2_F32 newLeft = portals[i].myLeft - apex;
            const H3_Vector2_F32 oldLeft = portals[leftIndex].myLeft - apex;
            const bool makesFunnel = GetVectorRelation(oldLeft, newLeft) == VectorRelation::Left;
            if (!makesFunnel)
            {
                const H3_Vector2_F32 rightSide = portals[rightIndex].myRight - apex;
                const bool crossesOtherSide = GetVectorRelation(rightSide, newLeft) == VectorRelation::Right;
                if (crossesOtherSide)
                {
                    outputPath.AddPoint(portals[rightIndex].myRight);
                    apex = portals[rightIndex].myRight;
                    // Find next portal where the right side is not our new apex.
                    bool newPortalFound = false;
                    for (uint j = rightIndex + 1; j < nOfPortals; j++)
                    {
                        if (portals[j].myRight != apex)
                        {
                            leftIndex = j;
                            rightIndex = j;
                            i = j;
                            newPortalFound = true;
                            break;
                        }
                    }

                    if (!newPortalFound)
                    {
                        // If we run out of portals then we can go straight to the end.
                        break;
                    }
                }
                else
                {
                    leftIndex = i;
                }
            }
        }

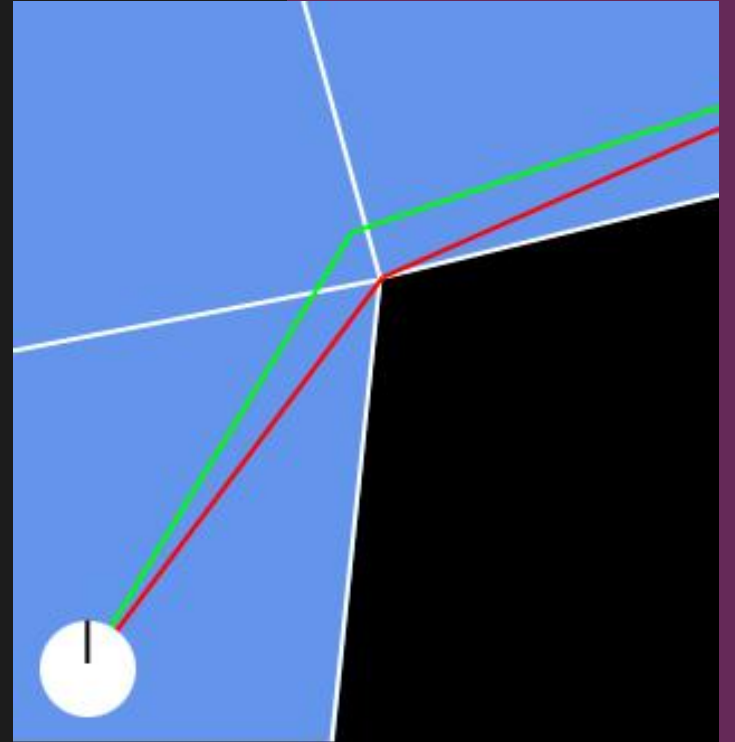
        if (portals[i].myRight != portals[rightIndex].myRight)
        {
            const H3_Vector2_F32 newRight = portals[i].myRight - apex;
            const H3_Vector2_F32 oldRight = portals[rightIndex].myRight - apex;
            const bool makesFunnel = GetVectorRelation(oldRight, newRight) == VectorRelation::Right;
            if (!makesFunnel)
            {
                const H3_Vector2_F32 leftSide = portals[leftIndex].myLeft - apex;
                const bool crossesOtherSide = GetVectorRelation(leftSide, newRight) == VectorRelation::Left;
                if (crossesOtherSide)
                {
                    outputPath.AddPoint(portals[leftIndex].myLeft);
                    apex = portals[leftIndex].myLeft;
                    // Find next portal where the left side is not our new apex.
                    bool newPortalFound = false;
                    for (uint j = leftIndex + 1; j < nOfPortals; j++)
                    {
                        if (portals[j].myLeft != apex)
                        {
                            leftIndex = j;
                            rightIndex = j;
                            i = j;
                            newPortalFound = true;
                            break;
                        }
                    }

                    if (!newPortalFound)
                    {
                        // If we run out of portals then we can go straight to the end.
                        break;
                    }
                }
                else
                {
                    rightIndex = i;
                }
            }
        }
    }

    outputPath.AddPoint(endPosition);
}
```

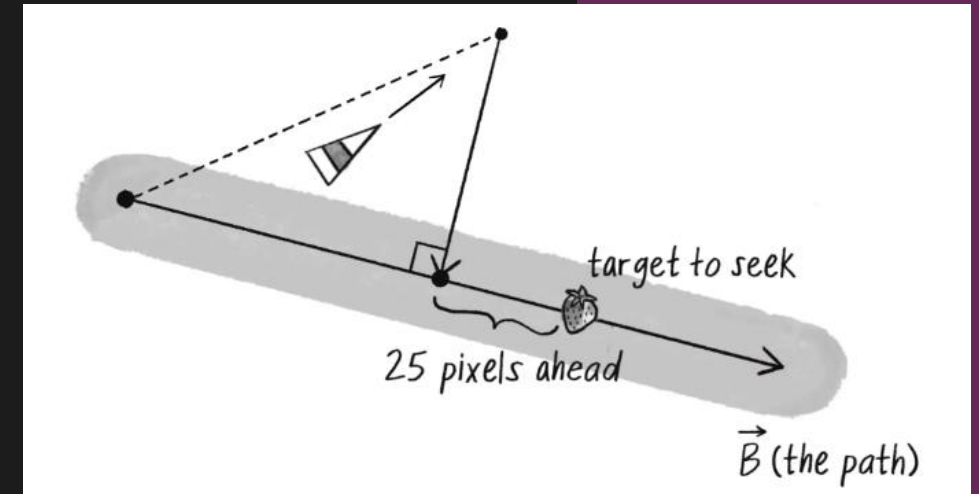
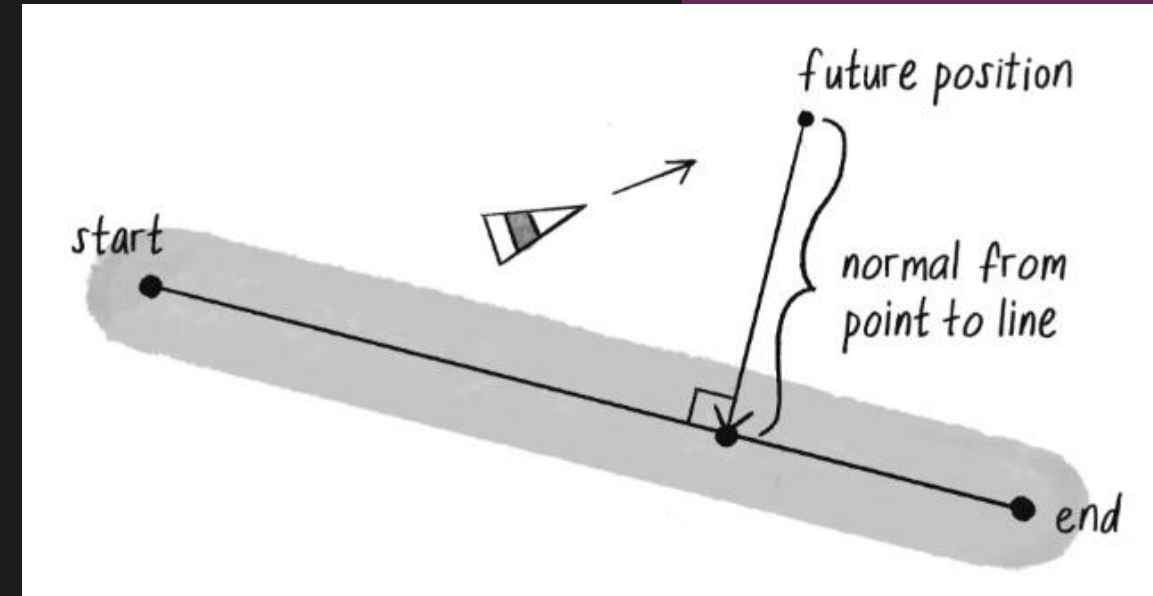
FUNNELING

- En grej med funneling är ju att den hittar den absolut kortaste vägen givet resultatet från vår A^* .
 - Dvs. den går exakt längs med väggar och genom hörn.
 - Detta kanske inte är önskvärt.
- För att rätta till detta finns det små ändringar man kan göra i sin funnel-algorithm som gör att man "knuffar" pathen ifrån hörnen.
- Men det går ju också att undvika antingen genom physics collisions eller wall avoidance.



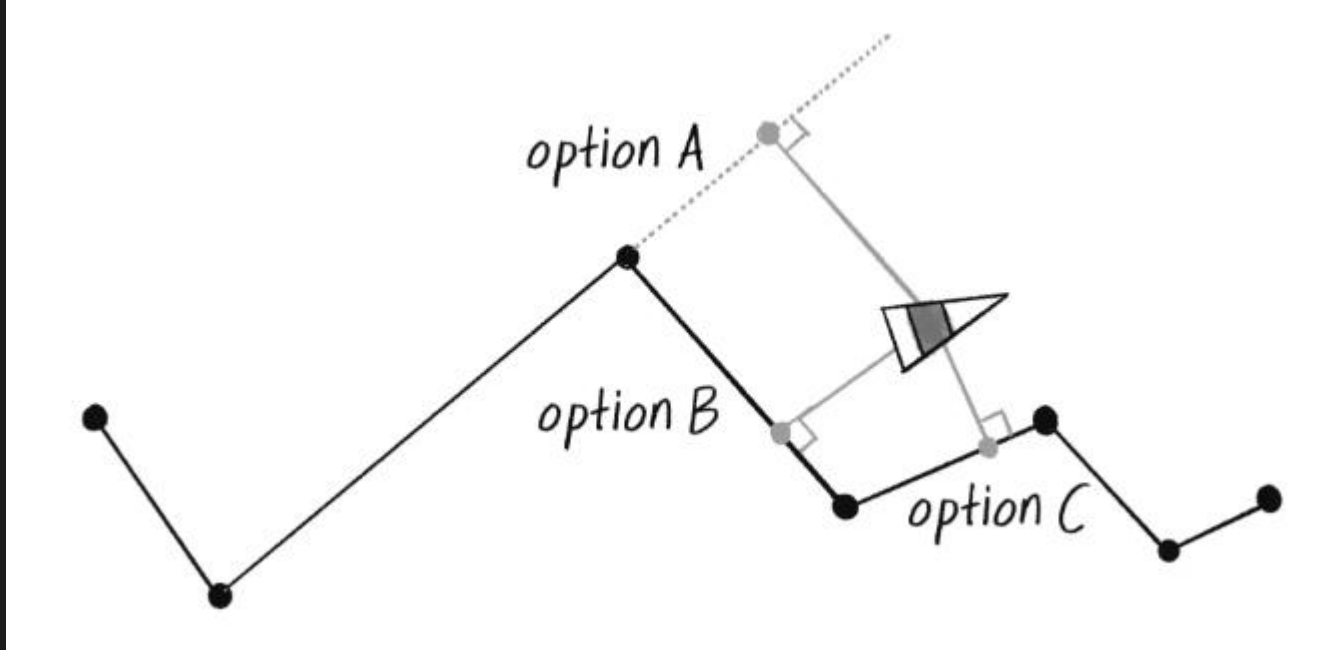
PATH FOLLOWING

- Det sista vi ska snacka om idag är path following.
 - Det är ett steering behaviour!
 - Om vi nu har pathfinding, vad kan då lämpa sig bättre att komplettera det med än path following?
- Detta är grunderna till path following:
 - Är min future position utanför en viss radie?
 - Om ja, då hittar vi den punkten på path:en som är närmst future position och seek:ar en punkt på pathen lite framför den.
 - Om nej, då fortsätter vi i vår nuvarande riktning.
- Vi har alltså tre tweakables
 - Vår path's "radie"
 - Avståndet till future position
 - Avståndet mellan normal point och target
- <https://natureofcode.com/autonomous-agents/#path-following>



PATH FOLLOWING

- Men vi har ju en path som består av mer än två punkter, hur väljer vi då normal point?



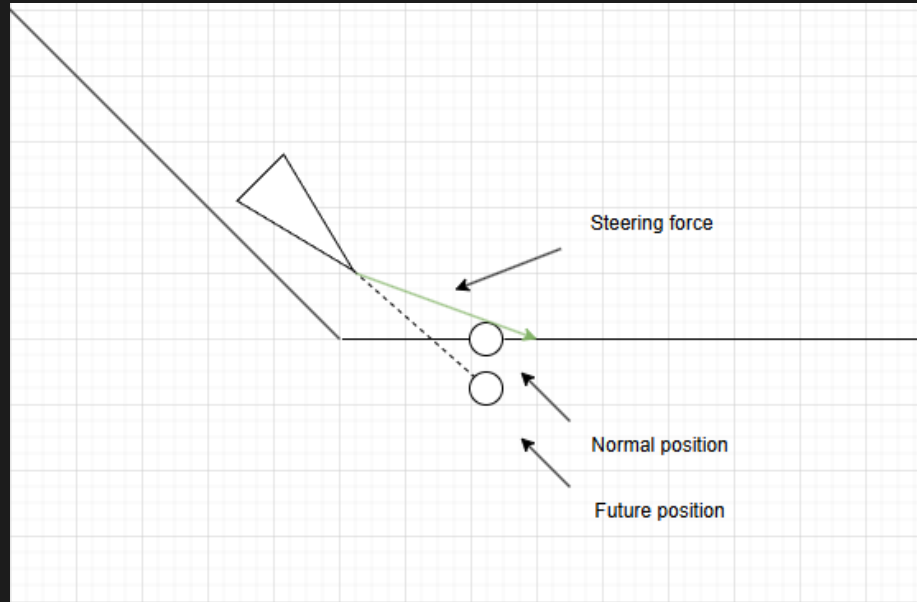
- Vi väljer den som är 1. På pathen, och 2. Närmst spelaren.
 - Dvs. alternativ C i bilden ovan

PATH FOLLOWING

- Värt att komma ihåg
 - Path following är bara en av många tekniker för att följa en path.
 - Generera en spline och följa den?
 - Eller hela tiden röra sig med konstant hastighet mot nästa punkt i pathen?
- Path following lämpar sig bra för AI, kanske inte lika bra för spelaren?
 - I P5 Production kanske ni t.ex. vill använda något mer specialiserat/hårdkodat.
 - Något som är enklare att kontrollera, debugga och tweakas.
 - Eller så funkar path following utmärkt!
 - Prova er fram om ni behöver.

PATH FOLLOWING

- En trevlig bi-effekt av path following är att den smoothar ut vår path.



- Så kanske behöver vi inte göra det där jag nämnde innan om att knuffa pathen ifrån hörn.
 - Eller så behöver vi båda!
 - Beror helt på spelet och på level layouten.
 - Ibland kanske det inte finns geometri där pathen "smoothas ut".

RESURSER

- Funneling

- <https://digestingduck.blogspot.com/2010/03/simple-stupid-funnel-algorithm.html>
- <https://ahamnett.blogspot.com/2012/10/funnel-algorithm.html>
- Men som sagt, finns inte så mycket online, och jag tycker båda dessa är ganska röriga. Kolla på min funneling på Omniway om ni vill istället, så kan vi diskutera den om ni har frågor.

- Path following

- Läs-format
 - <https://natureofcode.com/autonomous-agents/#path-following>
- Video-format
 - <https://www.youtube.com/watch?v=rIZYT-uvmGQ&list=PLRqwx-V7Uu6ZV4yEcW3uDwOgGXKUUSPOM&index=44>

- A* recap

- https://www.youtube.com/watch?v=-L-WgKMFuhE&list=PLFt_AvWsXI0cq5Umv3pMC9SPnKjfp9eGW&index=1
- En playlist i tio delar av Sebastian Lague.

KURSLITTERATUR

- Denna föreläsningen
- Nästa föreläsning
 - Chapter 4 - Pathfinding
 - 4.1 The Pathfinding Graph (s 195-203)
 - 4.2 Dijkstra (s 203-214)
 - 4.3 A* (s 215-237)
 - 4.4 World Representations (s 237-256)
- Nästa föreläsning
 - Chapter 5 - Decision Making
 - 5.1 Overview of Decision Making (s 297-299)
 - 5.2 Decision Trees (s 299-314)
 - 5.3 State Machines (s 314-337)

UPPGIFTEN

- Vi ska använda en navmesh för att begränsa var en AI kan gå.
- Det finns en navmesh (fbx-fil) på Omniway. Fritt fram att använda den, men det går även bra att läsa in en egen navmesh från Maya/Unity/Unreal.
 - Det enda kravet är att navmeshen ska läsas in (den får inte vara hårdkodad) och den måste vara tillräckligt "intressant".
 - Får inte vara en kvadrat med två trianglar.

UPPGIFTEN

- Ladda in er navmesh och rita ut alla noder.
- Det ska finnas två actors. En spelare och en companion.
- Spelaren styrs med ett arrive behaviour och rör sig dit man senast klickade på skärmen.
 - Spelaren är inte begränsad av navmeshen.
- Spelarens companion ska röra sig på navmeshen och röra sig till den punkten på navmeshen *som är närmast spelarens target*.
- Vägen som spelarens companion tar ska inte gå mellan varje triangels mittpunkt utan ska istället vara smooth:ad med något post process pass, t.ex. funneling.
- Rita ut både den o-smooth:ade och smooth:ade pathen er companion tar.
- Spelarens companion kan med fördel styras med steering behaviour Path Following, men det är inget krav.
- Allt som allt ska användaren kunna styra spelaren med mus-klick var som helst på skärmen och ha en companion som följer efter spelaren men som är begränsad till att röra sig på navmeshen.

UPPGIFTEN

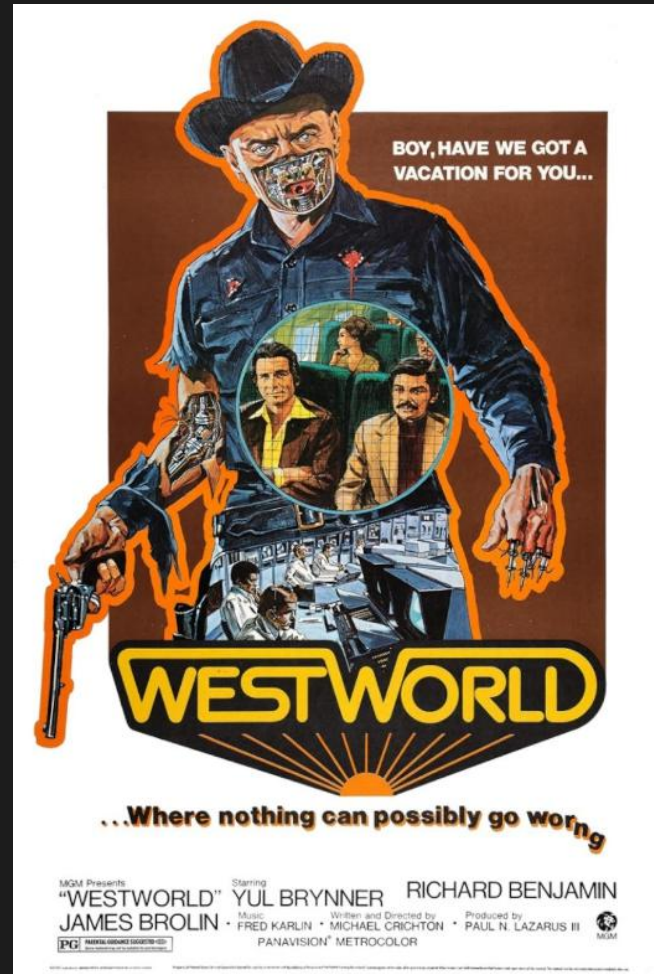
- Yikes, en hel del att göra!
- Fret not, följande finns på Omniway:
 - Kod för inladdning av navmesh (om ni väljer fbx-filen på Omniway)
 - Kod för funneling-algoritmen
- Så det finns en del frihet i denna uppgiften
 - Ni kan skapa en egen navmesh eller ta den som finns.
 - Ni kan koda er egna inladdning av navmesh eller ta den inladdningen som finns.
 - Ni kan koda er egna funneling, eller någon annan typ av path smoothing, eller ta funneling-koden som finns.
 - Ni kan välja att styra er companion med path following eller på annat valfritt sätt.
 - Det jag bryr mig om i denna uppgiften är resultatet och att ni använder er av en navmesh.
- Mer detaljer om uppgiften finns i beskrivningen på Omniway.

UPPGIFTEN

- Denna gången är inlämningen en code review!
 - När ni är klara, poke:a mig så kommer jag förbi och så kollar vi på resultatet tillsammans.
 - När jag säger att er lösning är godkänd så behöver ni bara "lämna in" en tom inlämning på Omniway så godkänner jag er där.
- Detta gör vi för att navmesh är en central del i P5_Production.
 - Och för att börja öva på code reviews
 - Mer code reviews kommer i P5_Production
- Viktigt att alla är med på hur navmesh fungerar.
 - Även om man inte är den som gör AI i gruppen så kan man behöva fixa buggar relaterade till navmesh, t.ex.
- (reminder to self: visa exempel)

DAGENS AI-FILMTIPS

- Westworld (1973), Michael Crichton



FRÅGOR?

